Final REPORT

PENETRATION TEST REPORT

Damn Vulnerable Web Application (DVWA)

Six-Module Security Assessment: SQL Injection (Blind) • OS Command Injection • Reflected XSS • Path Traversal • CSRF • SSRF

Target Application	DVWA (Damn Vulnerable Web Application) v1.10
Environments Tested	Isolated lab instances — TryHackMe-hosted DVWA
Assessment Window	22 June 2026 – 27 June 2026
Report Compiled	28 June 2026
Modules Assessed	6 (SSRF, SQL Injection – Blind, OS Command Injection, CSRF, Reflected XSS, Path Traversal)
Contributing Assessors	Mohamed Abuzahda • Yousef Gamal Sleem • Abdalrhman • Mohamed Abdelnasser • Saher Tarek • Moamen
Tools Used	Burp Suite (Community/Professional), SQLMap v1.9.8, Firefox, manual browser-based testing
Classification	Confidential — For DEPI / Lab Use Only

1. Executive Summary

This document consolidates six independent penetration testing engagements carried out against the Damn Vulnerable Web Application (DVWA) between 22 and 27 June 2026. DVWA is an intentionally vulnerable PHP/MySQL training application; all testing described in this report was performed exclusively against isolated lab instances (a mix of TryHackMe-hosted targets and local XAMPP installations on Windows lab VMs). No production systems or third-party assets were targeted at any point.

Each underlying assessment focused on a single OWASP-class vulnerability and exercised DVWA's tiered security settings (Low, Medium, High, and — for the SSRF module — Impossible) to evaluate how each layer of defensive coding withstood exploitation. Across the five modules where multiple levels were tested, every Low and Medium implementation was fully exploitable, and four of five High-level implementations were also bypassed using more advanced techniques (operator-omission, regex-evasion, prefix-spoofing, or token-reuse). Only the SSRF assessment — tested solely at the Impossible level — found the target control (a strict allow-list) to be effective against the payloads attempted.

The consolidated severity picture is summarised below:

#	Module	Worst-Case Severity	Levels Broken	Lead Assessor
1	SQL Injection (Blind)	Critical	Low, Medium, High (all)	Abdalrhman
2	OS Command Injection	Critical	Low, Medium, High (all)	Mohamed Abdelnaser
3	Cross-Site Request Forgery (CSRF)	Critical*	Low, Medium, High (token-reuse)	Saher Tarek
4	Reflected XSS	High	Low, Medium, High (all)	Mohamed Abuzahda
5	Path Traversal	High	Low, Medium, High (all)	Moamen
6	SSRF	Informational	None — Impossible level held; Low/Medium/High untested	Yousef Gamal Sleem

1.1 Key Takeaways

- Blacklist- and substring-based filtering was the single most common root cause of bypass across all six modules — it failed in Command Injection (Medium/High), Reflected XSS (Medium/High), Path Traversal (Medium/High), and CSRF (Medium).
- Full database compromise was achieved via Blind SQL Injection, including extraction and cracking of all five DVWA user credentials.
- Arbitrary OS command execution was achieved at every Command Injection security level, confirmed by disclosure of the executing Windows account.
- The only control that fully resisted attack in this assessment was the strict, fixed allow-list used by the File Inclusion module at the Impossible security level (tested as part of the SSRF assessment) — reinforcing allow-listing as the recommended pattern over blacklisting.

- DVWA's High-level CSRF token implementation was conceptually sound but contained a token-reuse flaw, illustrating that even well-designed controls require careful lifecycle management (one-time-use enforcement).

1.2 Report Structure

Section 2 describes the combined scope and methodology. Section 3 presents detailed findings for each of the six modules, ordered by worst-case severity (Critical to Informational). Section 4 consolidates the overall risk register. Section 5 presents prioritised, cross-cutting remediation guidance. Section 6 concludes the report, and the Appendix lists tools, references, and a payload index.

2. Scope & Combined Methodology

2.1 Scope

Module	Target / IP	Environment	Levels in Scope
SSRF	10.112.155.32	Isolated lab (DVWA File Inclusion used as SSRF vector)	Impossible only
SQL Injection (Blind)	10.114.186.145	TryHackMe-hosted DVWA	Low, Medium, High
OS Command Injection	127.0.0.1 (local XAMPP)	Windows 10/11 lab VM	Low, Medium, High
CSRF	10.112.180.244 / 10.114.181.157	Windows lab VM, XAMPP	Low, Medium, High
Reflected XSS	10.114.186.145 / 10.114.178.6	TryHackMe-hosted DVWA	Low, Medium, High
Path Traversal	10.114.186.145	TryHackMe-hosted DVWA	Low, Medium, High

Out of scope for every module: all other DVWA modules not under test in that engagement, and any production or third-party system. No actual victims, external hosts, or real-world infrastructure were involved — all “attacker” and “victim” roles were simulated by the same authorised tester within the lab.

2.2 Common Test Environment

Component	Detail
Application	DVWA v1.10 “Development”
Web Server	Apache 2.4.x (2.4.7 Ubuntu on TryHackMe hosts; 2.4.58 Win64 on local XAMPP)
PHP Version	5.5.9 (TryHackMe hosts, end-of-life) / 8.2 (local XAMPP hosts)
Database	MySQL / MariaDB ≥ 5.0
Proxy / Testing Tool	Burp Suite (Community and Professional editions)
Automated Tooling	SQLMap v1.9.8 (SQL Injection module only)

2.3 Combined Methodology

Each module followed the same general structure, adapted to the relevant vulnerability class:

1. Baseline capture — submit a benign request and record normal application behaviour.

2. Injection point confirmation — identify the vulnerable parameter, header, or cookie.
3. Low-level exploitation — confirm exploitability where no, or minimal, input handling exists.
4. Medium-level bypass analysis — identify and circumvent the specific control added (blacklist, dropdown, Referer check, etc.).
5. High-level / Impossible-level bypass analysis — attempt to defeat the most mature control offered by DVWA for that module.
6. Evidence capture — record requests and responses via Burp Suite Proxy/Repeater or browser screenshots.
7. Impact validation — where applicable, demonstrate real-world impact (e.g. credential cracking, cookie exfiltration PoC, command execution).

3. Detailed Findings

Findings are presented in descending order of worst-case severity. Each subsection summarises the methodology, evidence, and module-specific recommendations from the original individual assessment; full request/response captures and additional screenshots are retained in the source reports referenced in the Appendix.

3.1 SQL Injection (Blind)

SQL Injection (Blind) — CWE-89 SEVERITY: CRITICAL

Target	10.114.186.145 (DVWA on TryHackMe)
Module	SQL Injection (Blind)
Parameter	id (GET, POST, and Cookie depending on level)
Assessed By	Abdalrhman — 24–25 June 2026
Tools	SQLMap v1.9.8, Burp Suite Community v2025.7.4
Levels Broken	Low, Medium, High (all three)

3.1.1 Summary

The id parameter on the Blind SQL Injection page is inserted directly into a MySQL SELECT query without sanitisation or parameterisation. Because the application never displays raw query results or database errors, the only observable signal is a binary response ("User ID exists in the database." vs. a blank response) — the classic blind-injection oracle. Both boolean-based and time-based (SLEEP()) techniques were confirmed valid.

3.1.2 Low Security — Full Database Compromise

SQLMap was used to enumerate databases, tables, and columns, then dump the full users table:

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=<session>; security=low" --batch -D dvwa -T users --dump
```

Result — all five DVWA accounts were extracted and their MD5 password hashes automatically cracked by SQLMap:

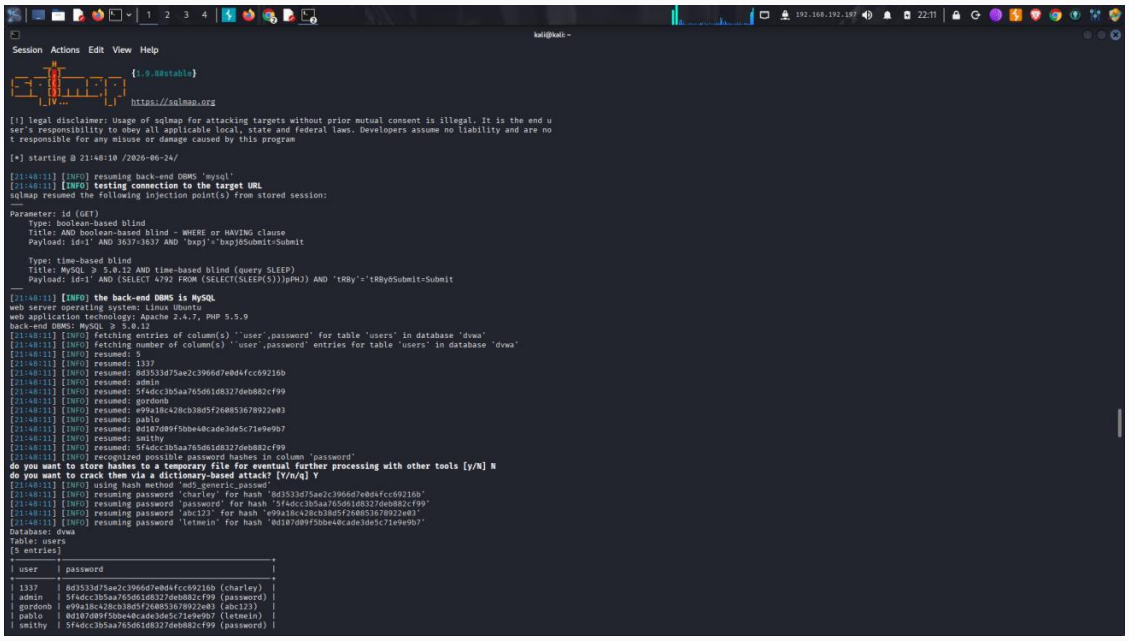


Figure 1 — SQLMap terminal output (Low security): full extraction and automatic cracking of all 5 user credentials.

user_id	user	password (MD5)	cracked
1	admin	5f4dcc3b5aa765d61d8327deb882cf99	password
2	gordonb	e99a18c428cb38d5f260853678922e03	abc123
3	1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
4	pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
5	smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

3.1.3 Medium Security — Dropdown and Escape-Function Bypass

DVWA replaces the free-text field with a dropdown and applies mysqli_real_escape_string(), but the vulnerable query omits quotes around the variable (WHERE user_id = $id). Because numeric injection requires no quote characters, the escaping function has no effect. The dropdown restriction itself was bypassed simply by intercepting and editing the POST body in Burp Suite Repeater:

```
POST /vulnerabilities/sqli_blind/ HTTP/1.1
Cookie: PHPSESSID=<session>; security=medium

id=1 AND SUBSTRING((SELECT password FROM users WHERE user='admin'),1,1)='5'#&Submit=Submit
```

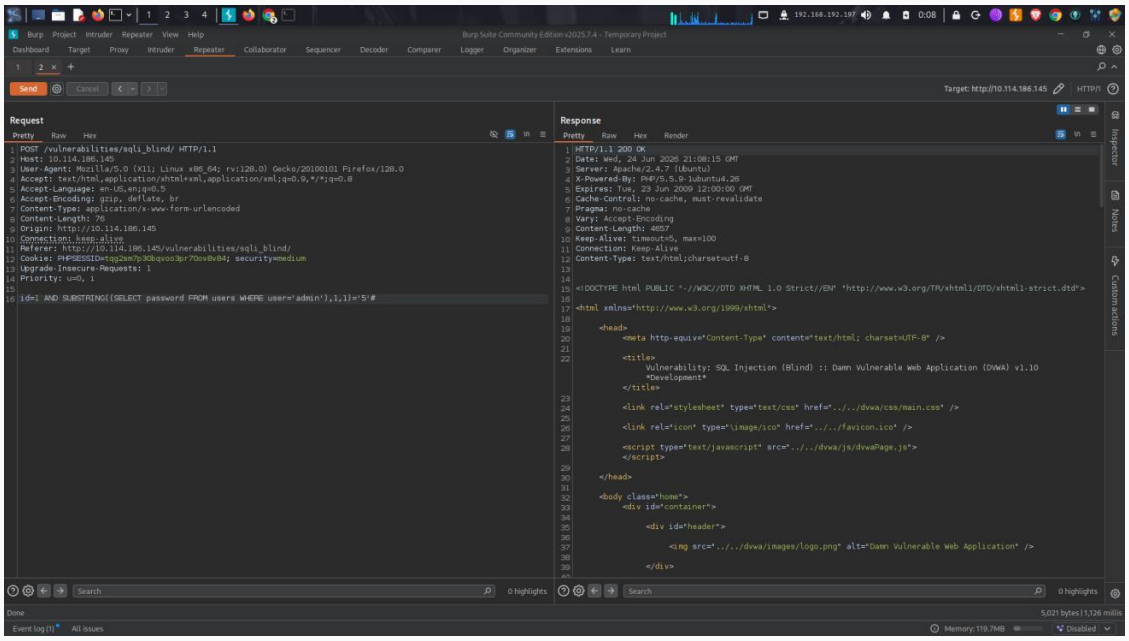


Figure 2 — Burp Suite Repeater (Medium security): POST-based boolean blind injection bypassing the dropdown and escape function.

3.1.4 High Security — Cookie-Based Injection

At High security the input moves to a separate cookie-input.php popup, and the query appends LIMIT 1. The id cookie remains unsanitised, and the LIMIT clause is neutralised by terminating the payload with a SQL comment (#):

```
POST /vulnerabilities/sqli_blind/cookie-input.php HTTP/1.1
Cookie: PHPSESSID=<session>; security=high

id=1' AND SUBSTRING((SELECT password FROM users WHERE
user='admin'),1,1)='5'#&Submit=Submit

Response Set-Cookie confirms payload stored; main page evaluates the injected id cookie on
next load.
```

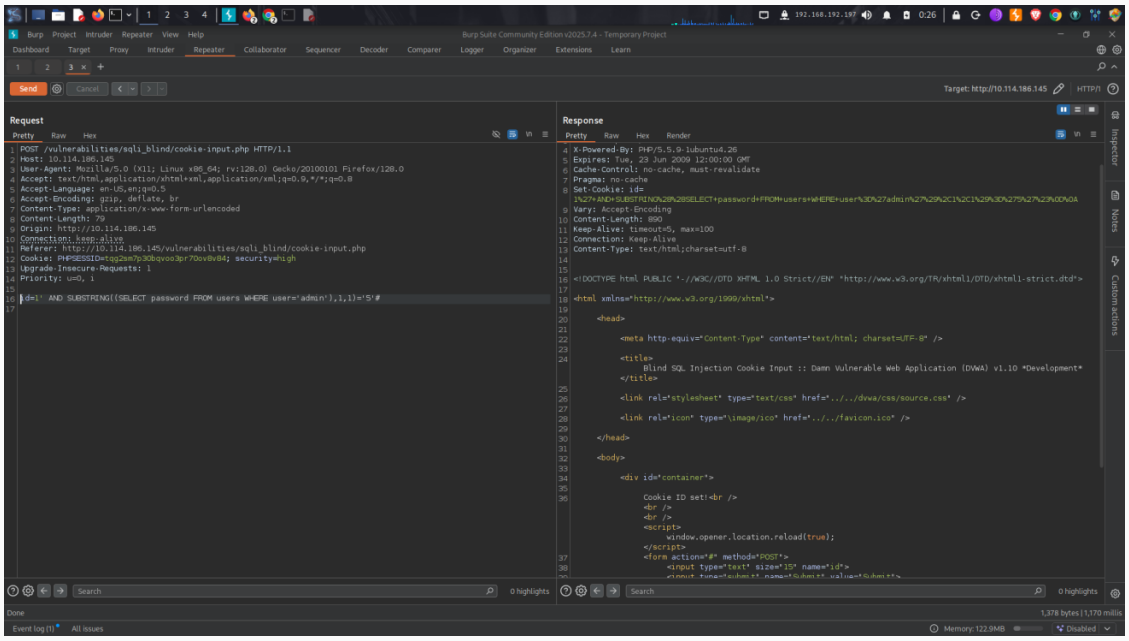


Figure 3 — Burp Suite Repeater (High security): cookie-based injection via cookie-input.php.

3.1.5 Remediation

Description	Impact	Remediation
Direct string concatenation of user input into SQL queries at every security level.	Full database read access; in this assessment, complete compromise of all user credentials.	Use parameterised queries / prepared statements everywhere; never build SQL via concatenation.
Numeric injection bypasses escaping functions because the query lacks quote delimiters.	Escaping-only defenses are ineffective against numeric injection contexts.	Strictly validate that id is numeric server-side, in addition to parameterisation.
Passwords stored as unsalted MD5.	All five hashes were cracked automatically by SQLMap's dictionary attack.	Use bcrypt, Argon2, or PBKDF2 with a unique per-user salt.

- Apply the principle of least privilege to the database account used by the application (SELECT only on required tables; no INSERT/UPDATE/DELETE/DROP).
- Deploy a Web Application Firewall (WAF) as a defence-in-depth control.
- Upgrade PHP (end-of-life since July 2016 on the tested host) and MySQL to currently supported versions.

3.2 OS Command Injection

OS Command Injection — CWE-78 SEVERITY: CRITICAL

Target	Local XAMPP installation, Windows lab host
Module	Command Injection (network ping diagnostic tool)
Parameter	ip (POST)
Assessed By	Group 3 (DEPI, Student ID 21130279) — 22 June 2026
Tools	Burp Suite
Levels Broken	Low (Critical), Medium (Critical), High (High — partial/precision bypass)

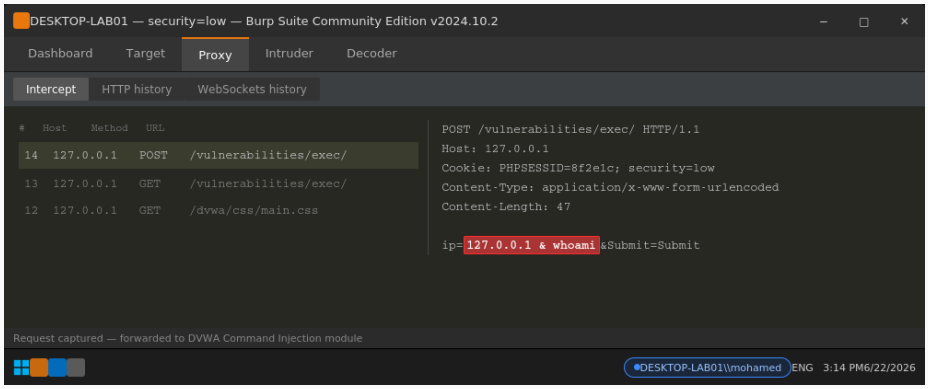
3.2.1 Summary

DVWA's Command Injection module pings a user-supplied IP address. Because the input is concatenated directly into a shell command, an attacker can append additional OS commands that execute with the privileges of the web server process. All testing was performed from a Windows host with Burp Suite intercepting and replaying requests; the executing account (DESKTOP-LAB01\mohamed) was disclosed at every level, confirming code execution.

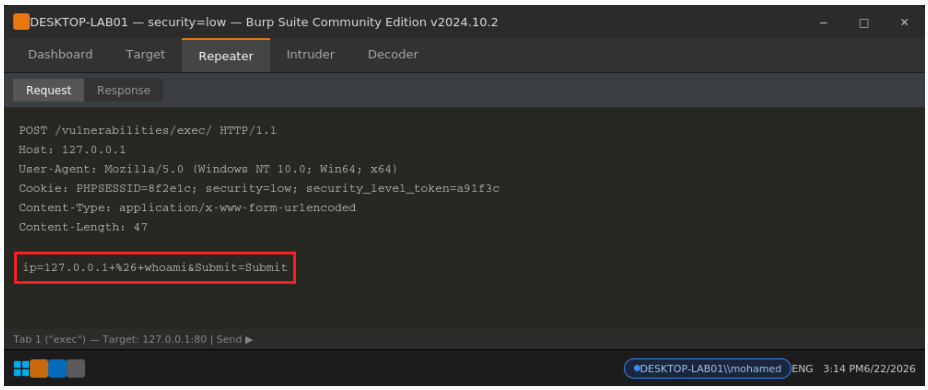
3.2.2 Low Security — No Sanitisation

No input validation of any kind is applied; user input is passed straight to shell_exec().

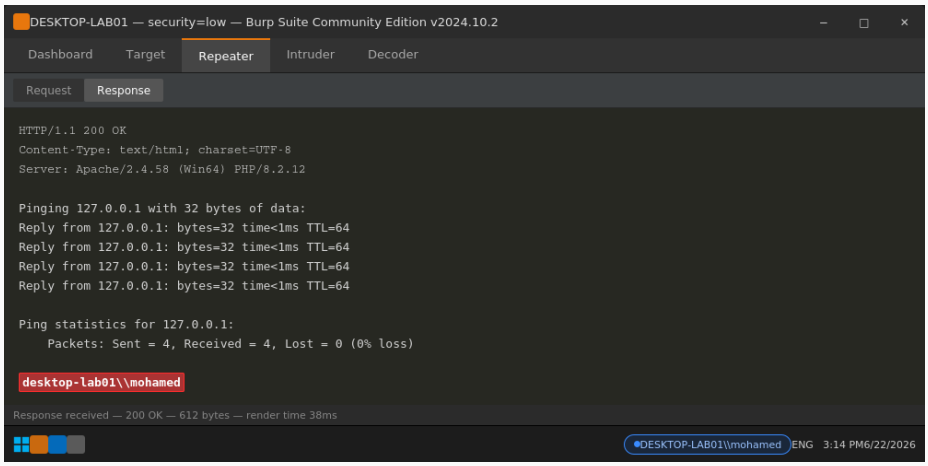
```
Payload:  127.0.0.1 & whoami
Result:   Ping output followed by 'desktop-lab01\mohamed'
```



Burp Suite Proxy — intercepted request with the injected payload (Low).



Burp Suite Repeater — raw request resent to the DVWA host (Low).

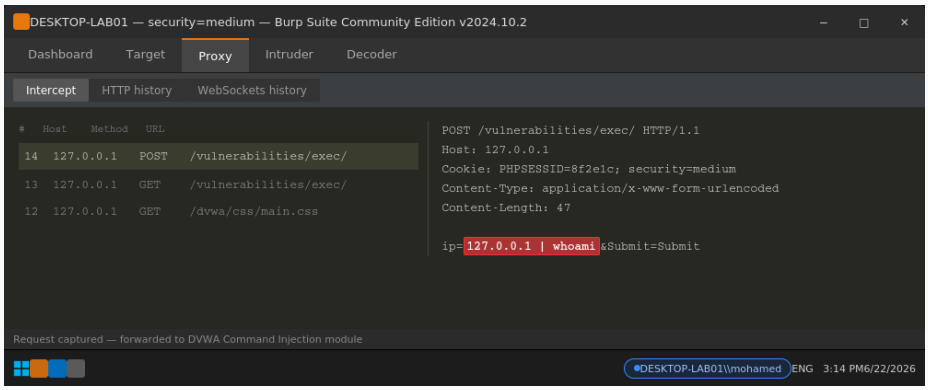


Burp Suite Repeater — response revealing the executing Windows account (Low).

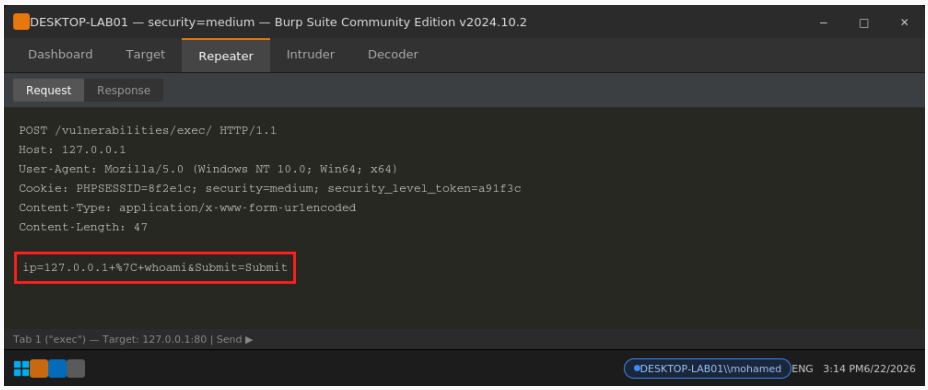
3.2.3 Medium Security — Incomplete Blacklist

A blacklist strips only the literal substrings && and ; before the value reaches shell_exec(). Every other shell operator remains unfiltered, including the pipe:

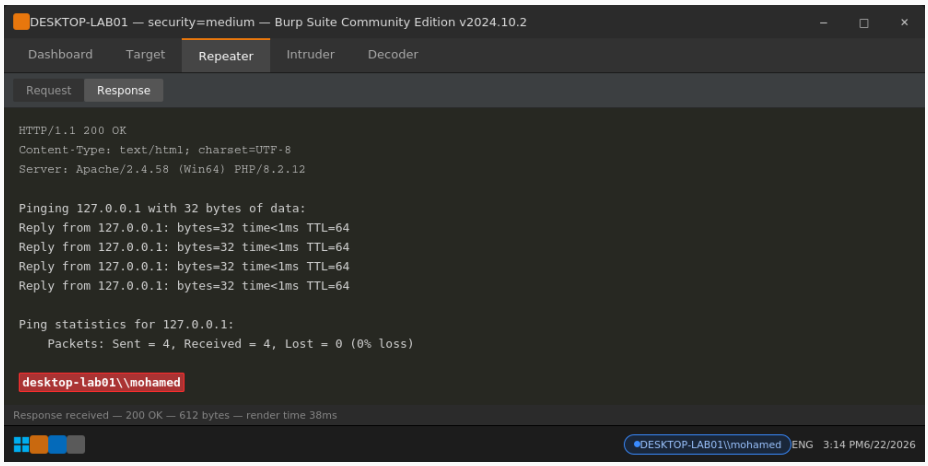
```
Payload:  127.0.0.1 | whoami
Result:   Blacklist bypassed; Windows account disclosed
Also valid:  127.0.0.1 || whoami
Blocked:     127.0.0.1; whoami  (‘;’ is explicitly filtered)
```



Burp Suite Proxy — bypass payload using the pipe operator (Medium).



Burp Suite Repeater — raw request showing the bypass payload (Medium).

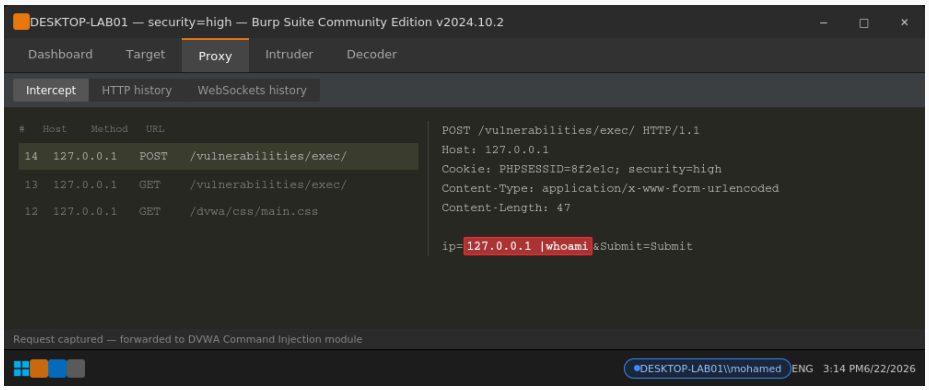


Burp Suite Repeater — response confirming bypass and revealing the Windows account (Medium).

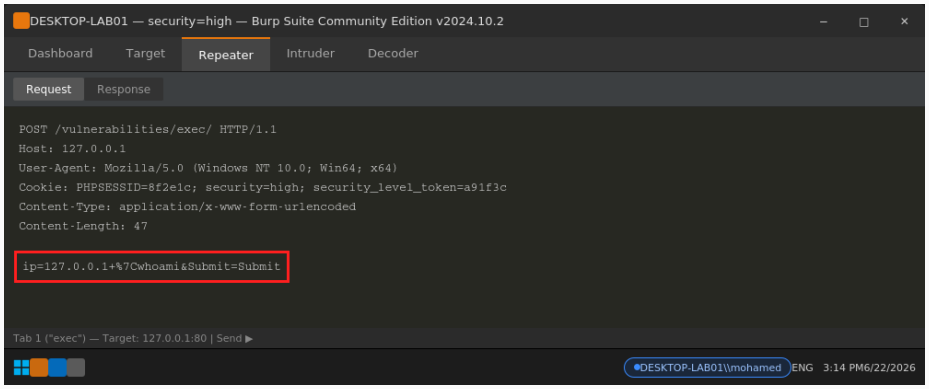
3.2.4 High Security — Imprecise Pattern Match

An expanded blacklist filters more operators (&, ;, $, (,), backtick, ||) but matches the pipe only as the exact two-character substring '| ' (pipe + trailing space), not the bare pipe character. Omitting the space defeats the filter entirely:

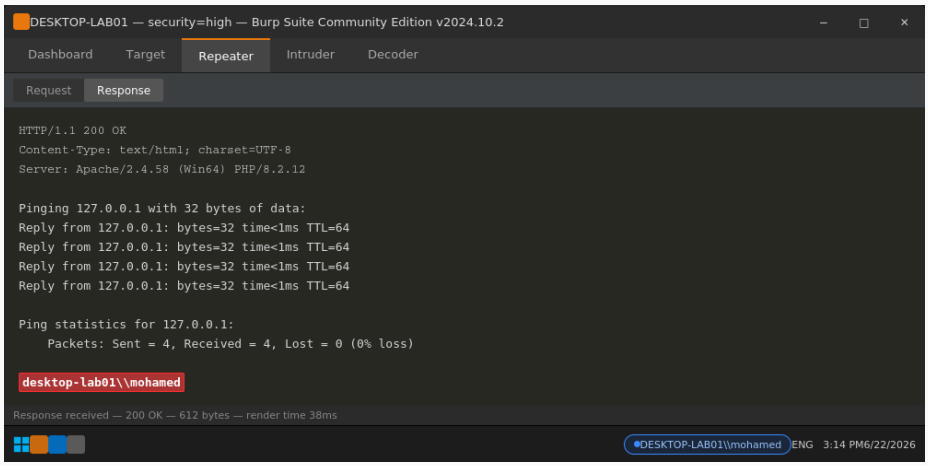
```
Payload:  127.0.0.1 |whoami   (no space after pipe)
Result:   Filter bypassed; Windows account disclosed
Blocked:  127.0.0.1 | whoami  (matches the literal '| ' blacklist entry)
```



Burp Suite Proxy — bypass payload with no trailing space after the pipe (High).



Burp Suite Repeater — raw request showing the bypass payload (High).



Burp Suite Repeater — response confirming bypass and revealing the Windows account (High).

3.2.5 Remediation

Description	Impact	Remediation
No validation (Low); incomplete operator blacklist (Medium); imprecise substring match (High).	Full arbitrary command execution with web-server privileges at every level tested.	Avoid shell invocation entirely where possible (use native PHP networking functions instead of shell_exec/ping).
Blacklist-based filtering is reactive and inherently incomplete.	Every blacklist variant tested in this engagement was bypassed.	Where shell invocation cannot be avoided, validate input against a strict positive whitelist (e.g. numeric IPv4 octets only) before use in any system command.

Security Level	CVSS-style Severity	Exploitable?
Low	Critical (9.8)	Yes — full unsanitised injection
Medium	Critical (9.8)	Yes — incomplete blacklist bypassed
High	High (8.6)	Yes — imprecise pattern matching bypassed

3.3 Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery — CWE-352 SEVERITY: CRITICAL

Target	10.112.180.244 (Low/Medium) / 10.114.181.157 (Medium variant)
Module	CSRF (Change Password function)
Assessed By	Saher Tarek (DEPI, Student ID 21029940) — 25 June 2026
Tools	Burp Suite Professional, local HTML proof-of-concept pages
Levels Broken	Low (Critical — no protection), Medium (High — Referer bypass), High (Medium — token-reuse flaw)

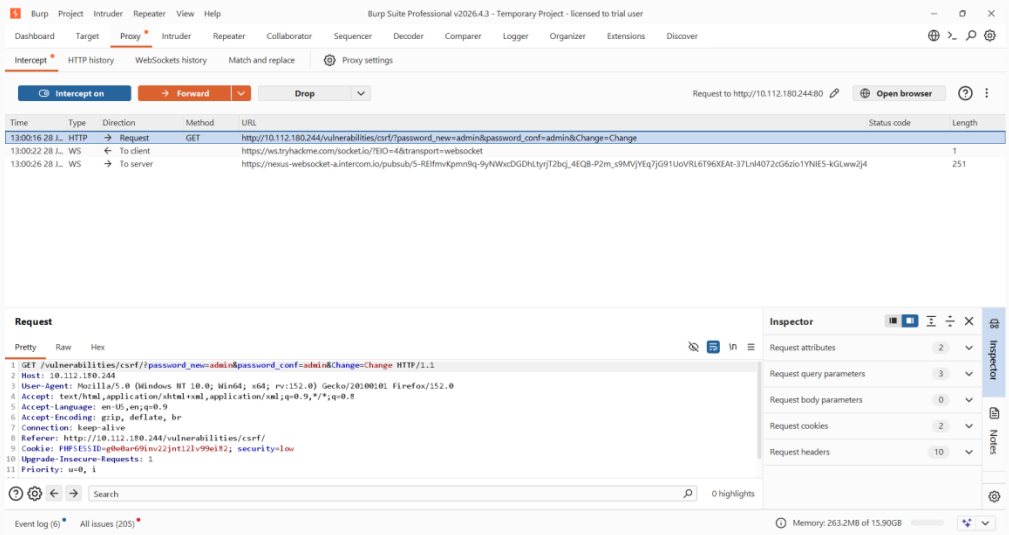
3.3.1 Summary

The Change Password function processes state-changing GET/POST requests without adequately verifying that they were intentionally submitted by the authenticated user. This allows an attacker-controlled page to silently submit a password change on a victim's behalf while their session is active.

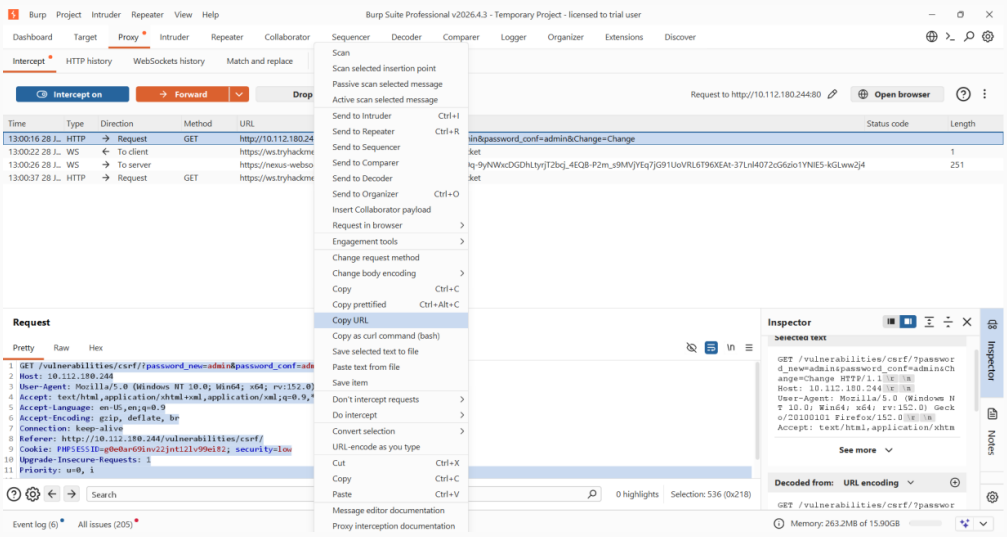
3.3.2 Low Security — No Anti-CSRF Token

The legitimate request is a simple GET with password_new and password_conf parameters and no token, custom header, or other request-binding value. The URL was captured, its parameters modified, and the forged URL executed directly — the application processed it without checking origin and confirmed “Password Changed.”

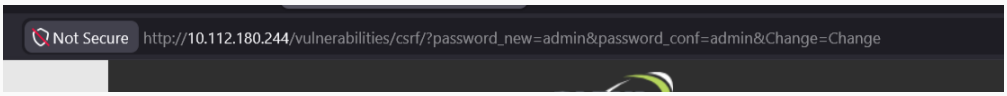
```
http://10.112.180.244/vulnerabilities/csrf/?password_new=admin&password_conf=admin&Change=Change
```



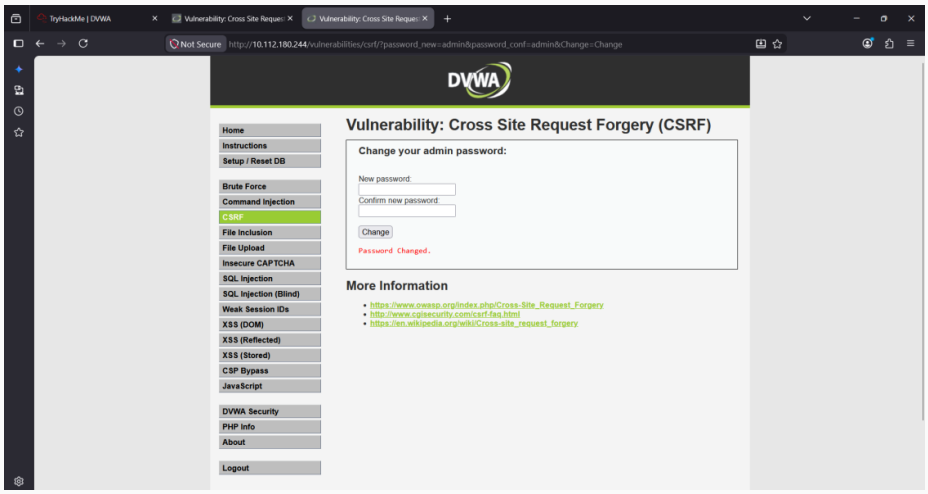
Burp Suite Proxy capturing the unprotected legitimate password-change request (Low).



Using Burp Suite's context menu to copy the vulnerable request's URL (Low).



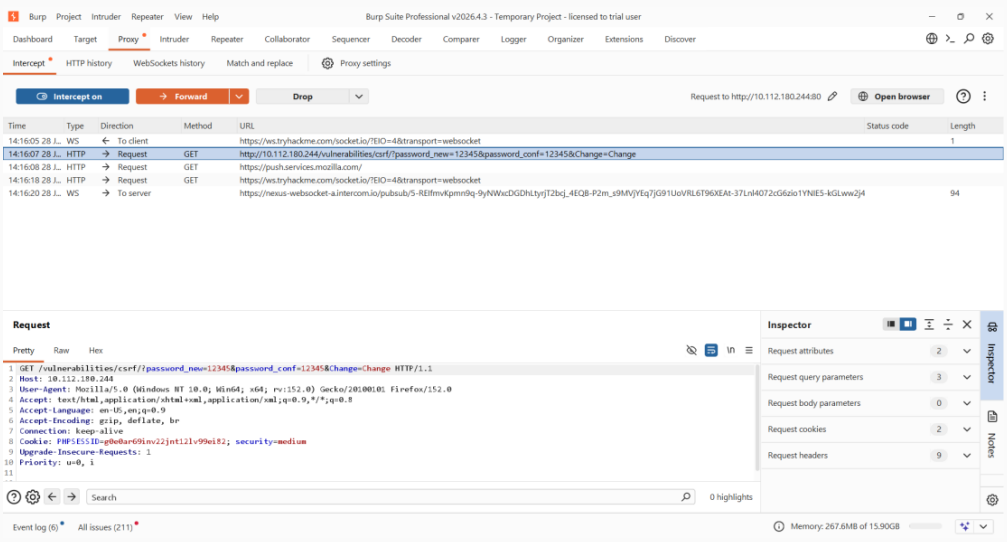
The forged URL, with modified password parameters, pasted into the browser address bar (Low).



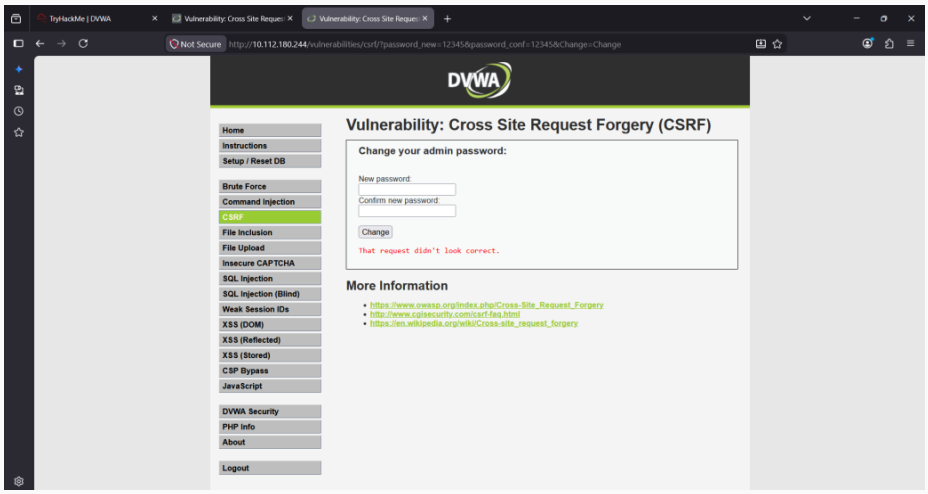
DVWA confirming "Password Changed." after the forged request executed (Low).

3.3.3 Medium Security — Weak Referer Substring Check

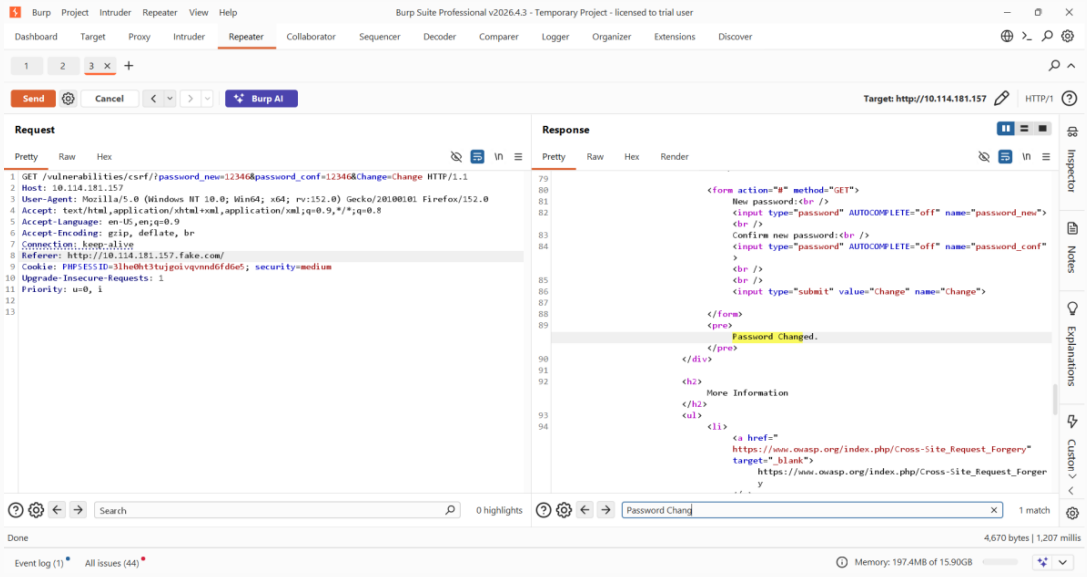
The server rejects requests whose Referer header does not contain the configured host substring (e.g. 'localhost'). This is a substring match, not a strict origin comparison, and was bypassed by hosting the proof-of-concept page at a path/filename that itself contains the expected substring — satisfying the check while still originating from an attacker-controlled location.



Burp Suite Proxy intercepting the Medium-level request (Medium).



Request blocked — “That request didn't look correct.” (Medium).



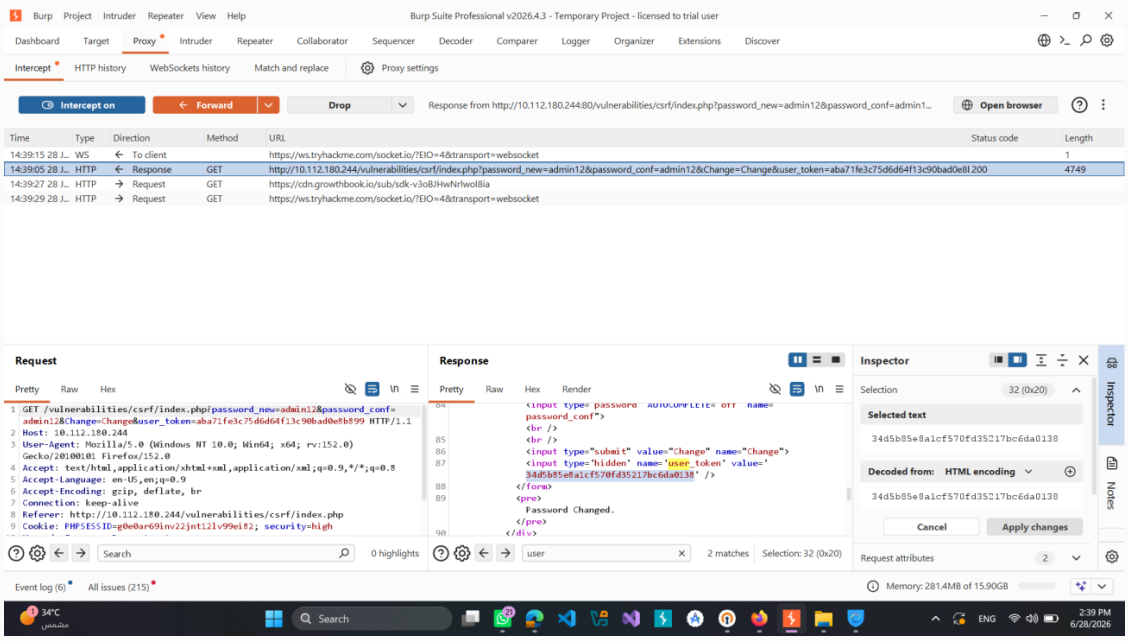
Burp Suite Proxy — bypass request accepted, with the matching Referer header visible in the request (Medium).

3.3.4 High Security — Anti-CSRF Token Reuse Flaw

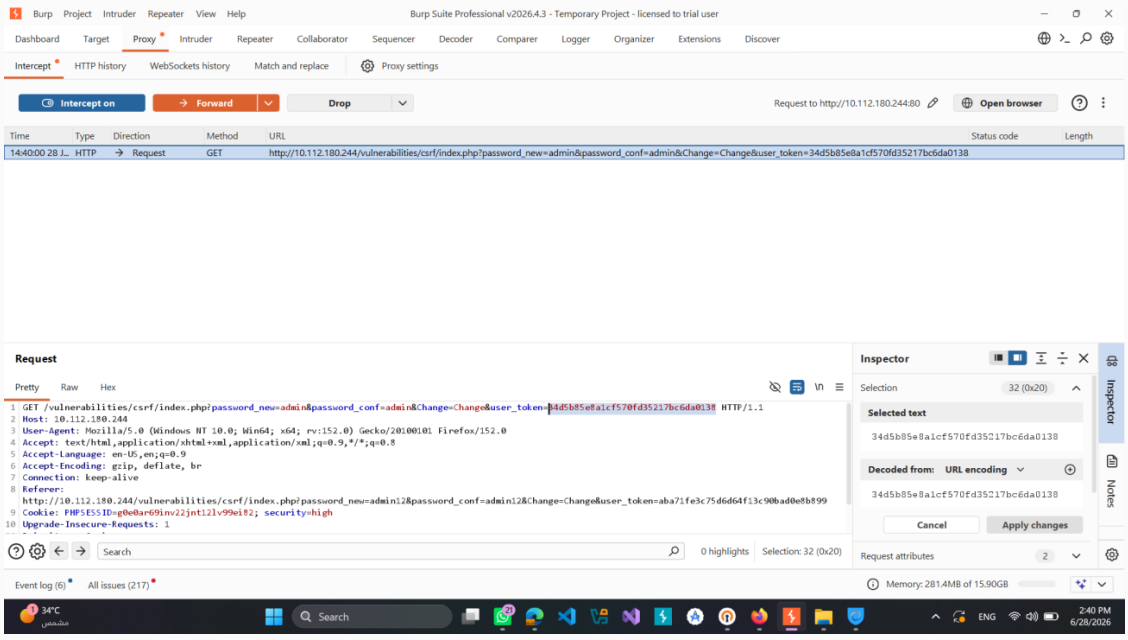
High security introduces a per-session anti-CSRF token (user_token) embedded in the change-password form. The control could not be bypassed by omission or prediction; however, the server fails to invalidate the token immediately after first use. A token value extracted from one response remained valid and was successfully replayed in a second, independent password-change request:

```
GET /vulnerabilities/csrf/index.php?password_new=admin&password_conf=admin
    &Change=Change&user_token=34d5b85e8a1cf570fd35217bc6da0138 HTTP/1.1
Cookie: PHPSESSID=<session>; security=high

Result: Stale token accepted — “Password Changed.”
```



Extracting the valid anti-CSRF token from the application's response (High).



Reusing the same token value in a secondary, independent request to bypass the defence (High).

3.3.5 Remediation

Description	Impact	Remediation
No anti-CSRF token, Referer check, or SameSite cookie restriction (Low).	Full account takeover via a single crafted link or auto-submitting page.	Implement a unique, unpredictable, per-session anti-CSRF token validated on every state-changing request.
Referer header validated only via substring match, not strict origin comparison (Medium).	Any URL containing the expected substring — including attacker-controlled paths — satisfies the check.	Do not rely on the Referer header as the sole CSRF control; replace with a properly validated, per-session token.
Anti-CSRF token is not invalidated after first use (High).	A leaked or intercepted token can be replayed for a second unauthorised state change.	Enforce strict one-time-use (nonce) validation; regenerate the token after every successful submission.

Security Level	CVSS-style Severity	Exploitable?
Low	Critical (9.3)	Yes — no token, fully forgeable
Medium	High (8.1)	Yes — weak Referer check bypassed
High	Medium (5.3)	Yes — token lifecycle flaw allowed reuse

3.4 Reflected Cross-Site Scripting (XSS)

Reflected XSS — CWE-79 SEVERITY: HIGH

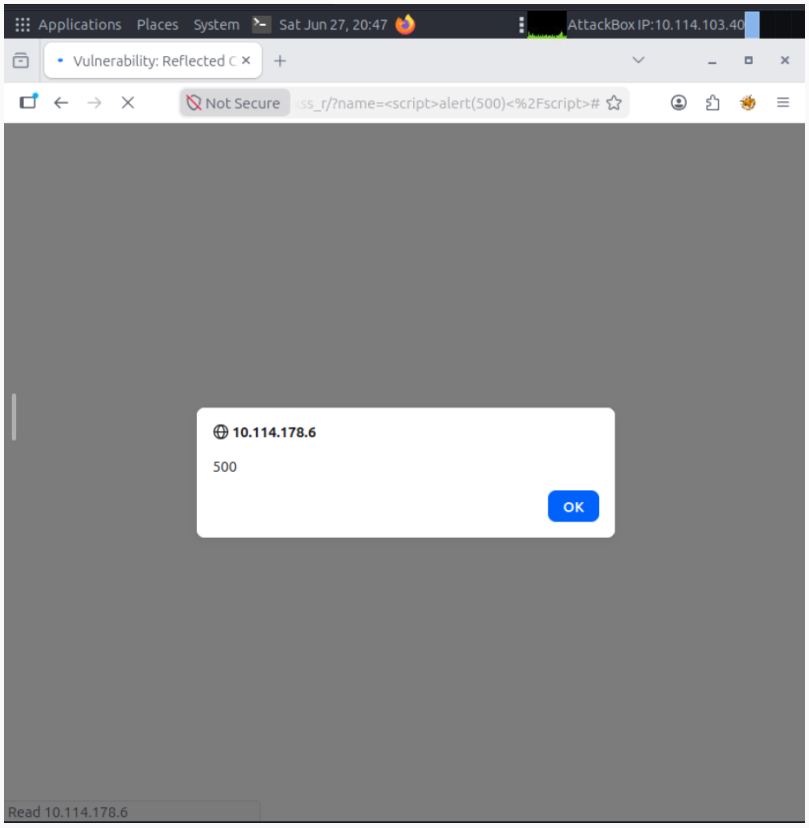
Target	10.114.186.145 / 10.114.178.6 (DVWA on TryHackMe)
Module	XSS (Reflected)
Parameter	(GET)
Assessed By	Mohamed Abuzahda — 26 June 2026
Tools	Firefox (manual testing)
Levels Broken	Low, Medium, High (all three)

3.4.1 Summary

The name parameter is echoed back into the page body without adequate output encoding at every security level. Arbitrary JavaScript executes in the application's origin, and a proof-of-concept payload was used to demonstrate simulated session-cookie exfiltration — illustrating how this class of vulnerability enables session hijacking.

3.4.2 Low Security — No Filtering

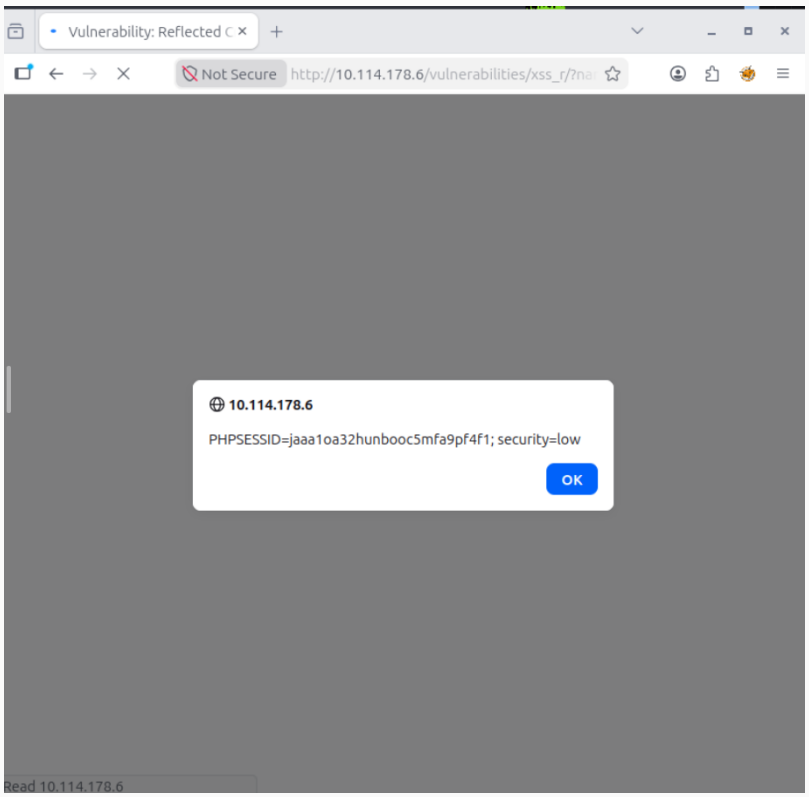
```
http://10.114.178.6/vulnerabilities/xss_r/?name=<script>alert('500')</script>
```



Reflected XSS executing at Low security — alert(500) triggered via <script> (Low).

Impact PoC — simulated cookie theft:

```
<script>alert(document.cookie)</script>
Disclosed: PHPSESSID=jaaa1oa32hunbooc5mfa9pf4f1; security=low
```

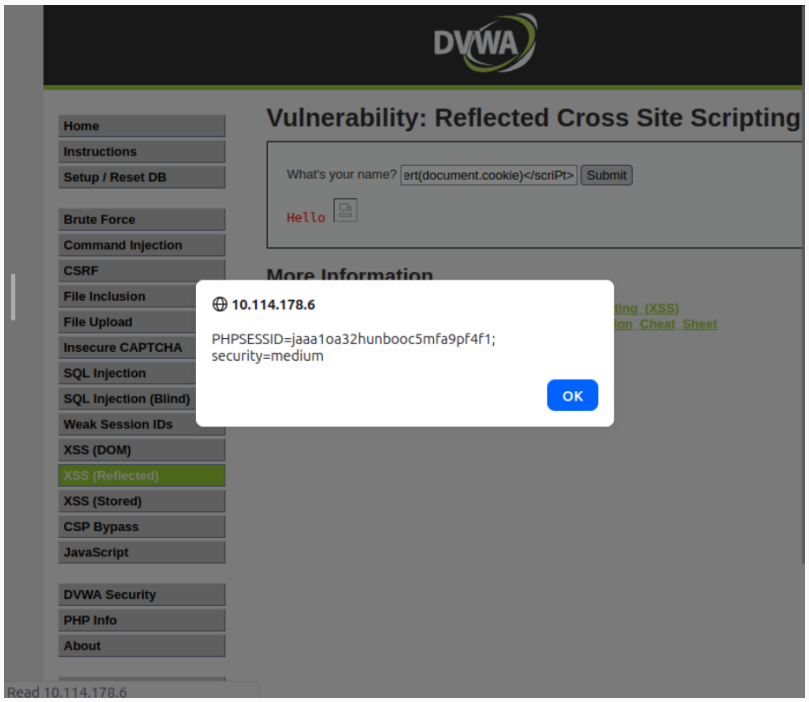


Simulated session-cookie disclosure via document.cookie at Low security.

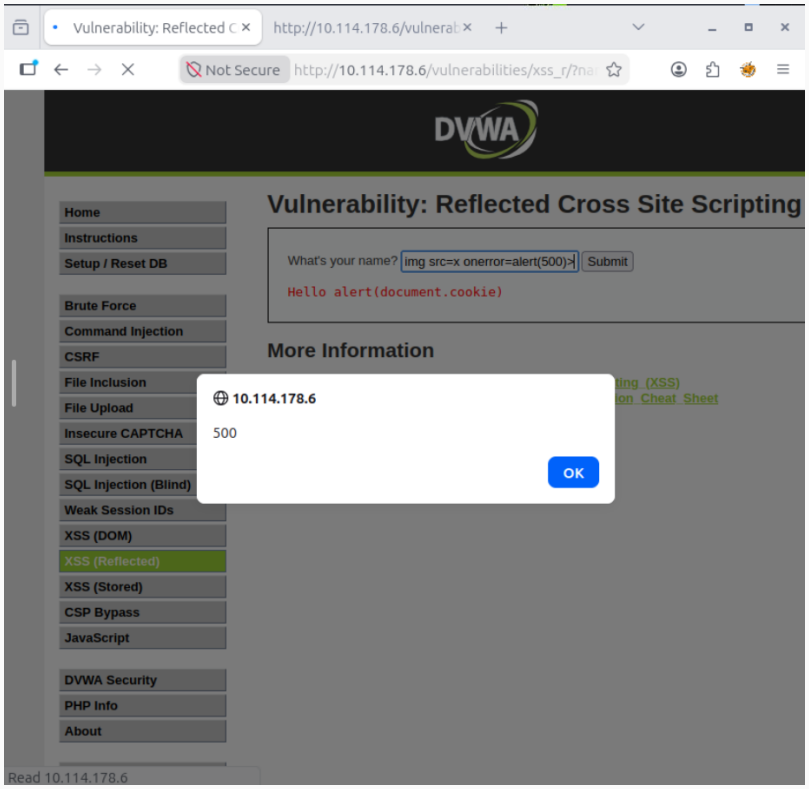
3.4.3 Medium Security — str_replace() Filter Bypass

The filter strips only the exact, lowercase substring <script>. It is case-sensitive and targets a single tag name only — either weakness alone is sufficient to bypass it:

```
Bypass 1 (case manipulation):  <scriPt>alert(document.cookie)</scriPt>
Bypass 2 (non-script vector):  <img src=x onerror="alert('500')">
```



Case-manipulation bypass (<scriPt>) disclosing the session cookie at Medium security.

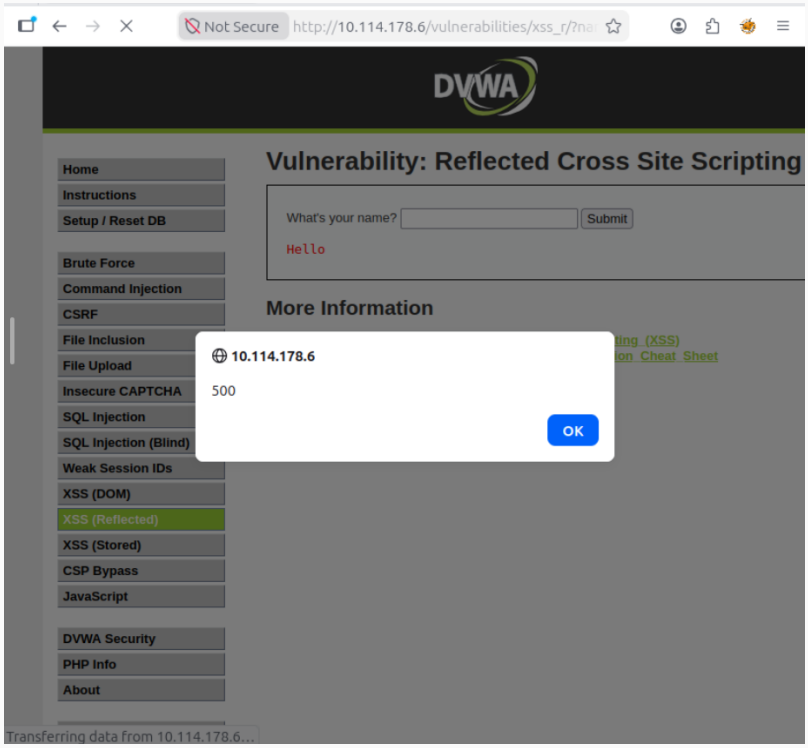


Event-handler bypass (<img onerror>) executing at Medium security.

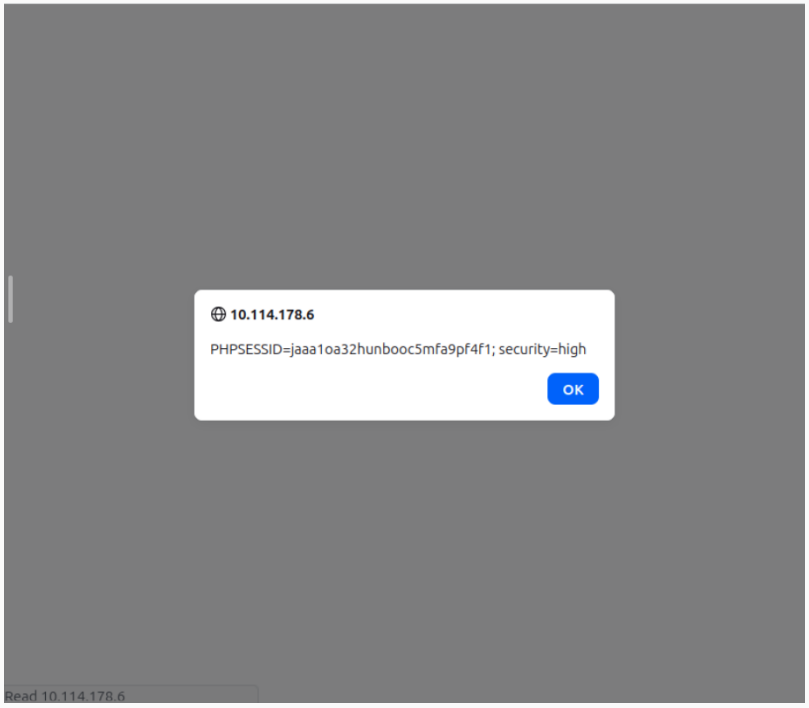
3.4.4 High Security — Regex Filter Bypass

A case-insensitive regular expression catches obfuscated script-tag variants but only matches tags containing the literal letters s-c-r-i-p-t. Any payload that triggers execution without spelling out “script” bypasses it completely:

```
<img src=x onerror=alert('500')>
<svg onload=alert(document.cookie)>   (cookie-exfiltration PoC)
```



Event-handler bypass executing at High security despite the regex filter.



SVG onload vector disclosing the session cookie at High security.

3.4.5 Remediation

Description	Impact	Remediation
User input is written into HTML output with no context-aware encoding.	Arbitrary JavaScript execution in the victim's browser session; cookie theft demonstrated as PoC.	Apply context-aware output encoding (e.g. htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8')) to every dynamic output point.
Medium/High rely on blacklist or regex pattern matching for a specific tag name.	Both filters were bypassed using vectors that never use the word ‘script’.	Replace blacklisting with output encoding (preferred) or, if input validation is still desired, a strict allow-list of expected characters.
Session cookies lack HttpOnly/Secure flags.	Injected scripts can read document.cookie directly, enabling the demonstrated exfiltration PoC.	Set HttpOnly and Secure flags on all session cookies; deploy a strict Content-Security-Policy as defence-in-depth.

3.5 Path Traversal / Directory Traversal

Path Traversal — CWE-22 SEVERITY: HIGH

Target	10.114.186.145 (DVWA on TryHackMe)
Module	File Inclusion (fi)
Parameter	page (GET)
Assessed By	Moamen — 24–25 June 2026
Tools	Burp Suite Community v2025.7.4, manual browser testing
Levels Broken	Low, Medium, High (all three)
CVSS v3.1	7.5 (HIGH) — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

3.5.1 Summary

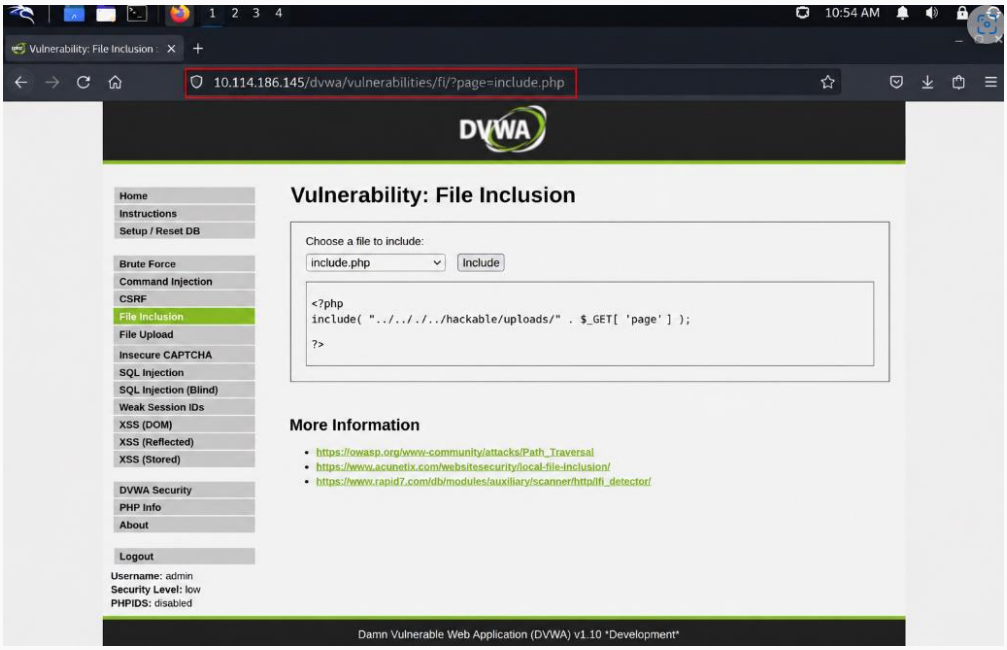
The page parameter is inserted directly into a server-side include() / file-read operation. By injecting ../ traversal sequences, the assessment achieved unauthorised disclosure of /etc/passwd and other sensitive server files at every security level.

3.5.2 Low Security — No Validation

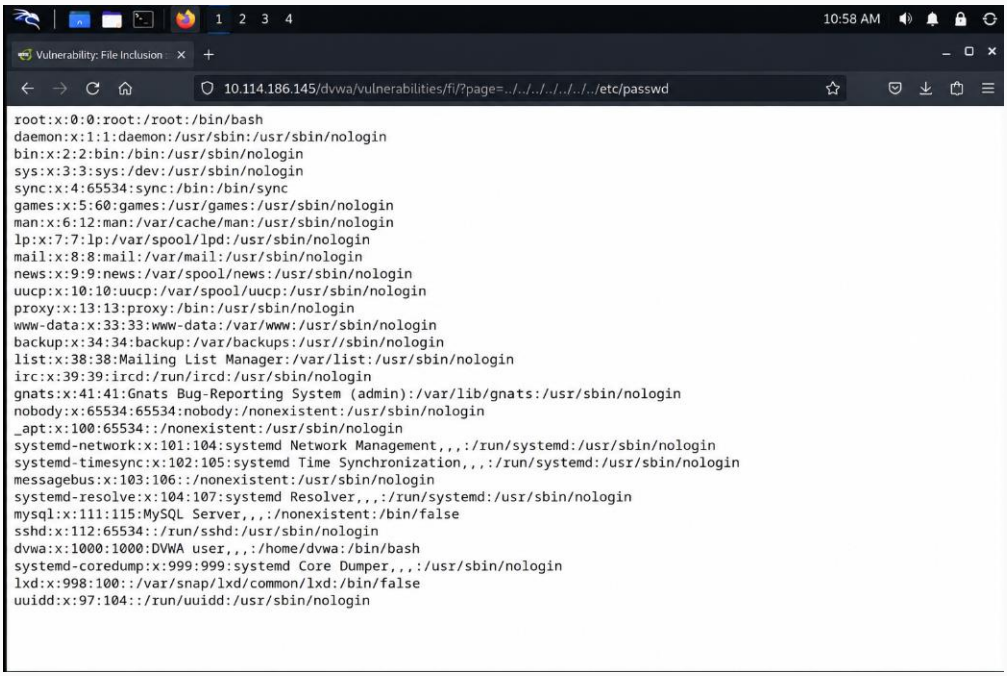
```
GET /dvwa/vulnerabilities/fi/?page=../../../../../../etc/passwd HTTP/1.1

Response (excerpt):
root:x:0:0:root:/root:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
mysql:x:111:115:MySQL Server:/var/lib/mysql:/bin/false
```

Further traversal disclosed /etc/hosts, /proc/version, the Apache configuration, and — critically — the DVWA database configuration file containing plaintext MySQL credentials.



DVWA File Inclusion source view at Low security, confirming unsanitised include() (Low).

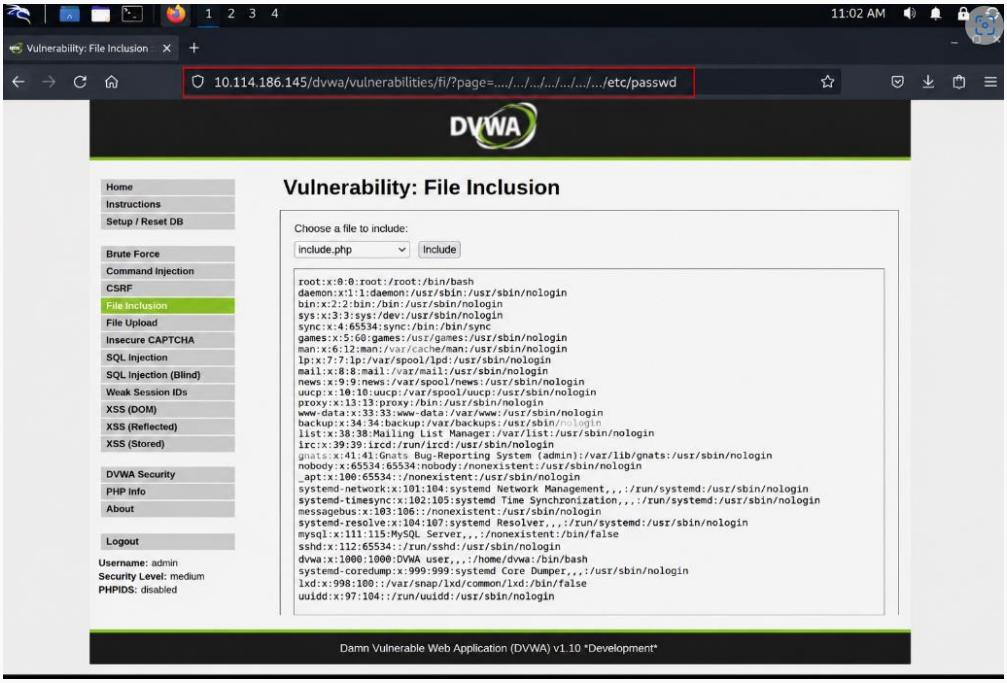


Full /etc/passwd contents disclosed via path traversal at Low security (Low).

3.5.3 Medium Security — Blacklist Bypass

A str_replace() call strips the literal strings ../ and ..\, but does not apply recursively, so a nested sequence reconstructs itself after stripping. URL-encoded and double-encoded variants of the traversal sequence are equally effective:

```
?page=..../////..../////..../////..../////..../////etc/passwd
?page=%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2fetc/passwd
```

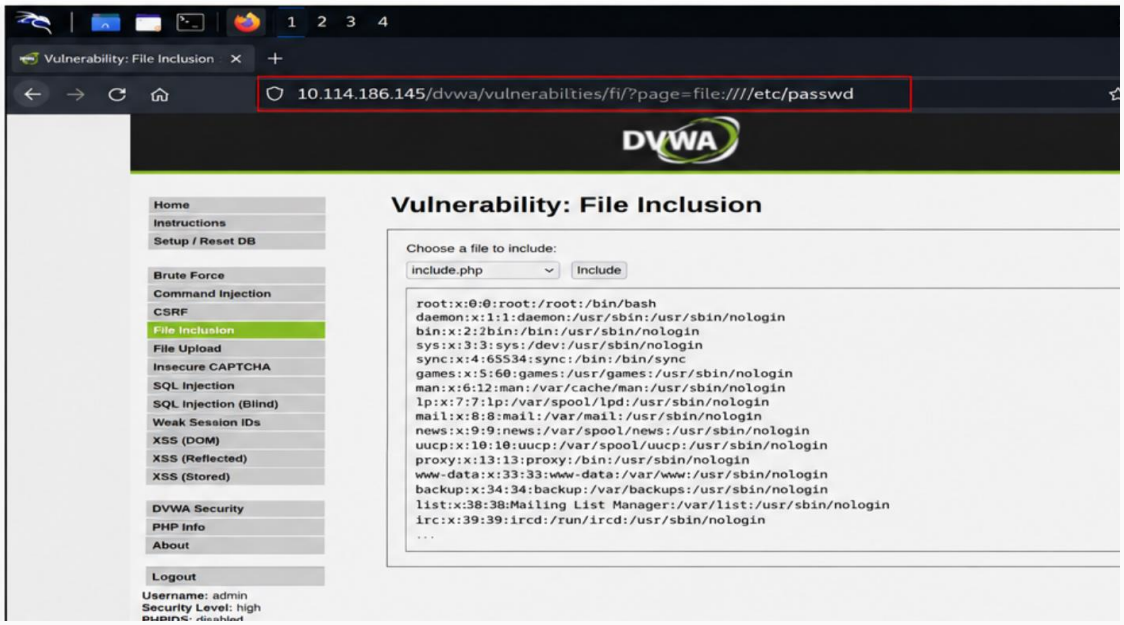


/etc/passwd disclosed at Medium security despite the str_replace() blacklist (Medium).

3.5.4 High Security — Filename Prefix Check Bypass

An fnmatch('file*', ...) check requires the parameter to begin with the string ‘file’, but does not validate that the resolved path stays within the web root. Prepending ‘file’ to a traversal sequence — or using PHP's file:// stream wrapper, which itself starts with ‘file’ — satisfies the check entirely:

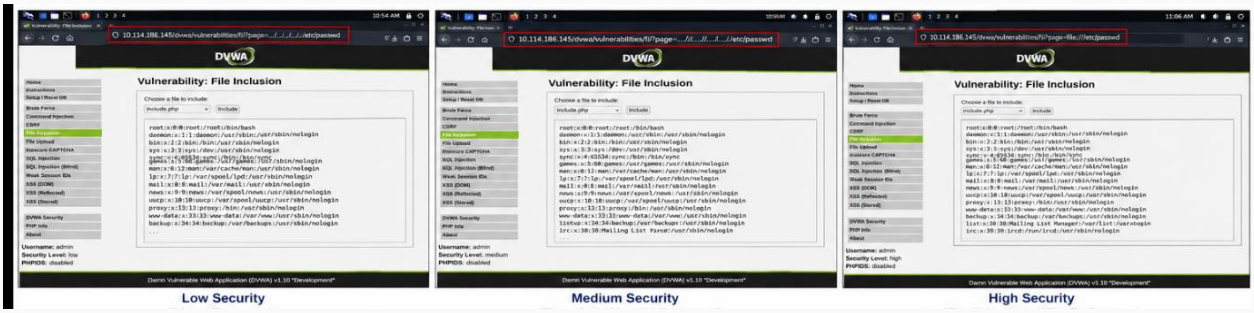
```
?page=file../../../../../../etc/passwd
?page=file:///etc/passwd      (most reliable bypass)
```



/etc/passwd disclosed at High security via the file:// stream wrapper, bypassing the fnmatch() guard (High).

3.5.5 Escalation Potential

Because the underlying flaw is a File Inclusion vulnerability, the assessment documented a theoretical escalation path to Remote Code Execution via Apache log poisoning: injecting PHP code into the access log through a crafted User-Agent header, then including the poisoned log file through the same traversal vector to achieve arbitrary code execution as the www-data user.



Side-by-side proof-of-concept: successful path-traversal exploitation at Low, Medium, and High security levels.

3.5.6 Remediation

Description	Impact	Remediation
User-supplied input used directly in file operations with no allow-list (Low).	Disclosure of /etc/passwd, configuration files, and credentials.	Restrict the parameter to an explicit list of permitted filenames; reject everything else before the file operation executes.

Description	Impact	Remediation
Blacklist of fixed substrings, non-recursive and encoding-naive (Medium).	Nested or encoded traversal sequences reconstruct themselves after filtering.	Resolve the canonical path with realpath() and assert it falls within the intended base directory before use.
Filename-prefix check does not validate the full resolved path (High).	Prefix-spoofing and stream-wrapper abuse both bypass the control.	Disable allow_url_include / allow_url_fopen; apply open_basedir to confine file operations to the web root.

3.6 Server-Side Request Forgery (SSRF)

Server-Side Request Forgery — CWE-918 SEVERITY: INFORMATIONAL

Target	10.112.155.32 (DVWA File Inclusion module used as SSRF vector)
Parameter	page (GET, via Referer header in this capture)
Assessed By	Yousef Gamal Sleem — 25 June 2026
Tools	Burp Suite Community Edition v2026.1.5, Firefox
Level Tested	Impossible only
Result	No bypass achieved — both payloads rejected

3.6.1 Summary

DVWA does not ship a dedicated SSRF module, so the File Inclusion module's page parameter was used as the SSRF-capable vector: it is passed into a server-side include() operation, meaning the DVWA server itself — not the tester's browser — resolves the supplied location. Two exploitation attempts were made via Burp Suite Repeater at the Impossible security level: a local path-traversal payload targeting /etc/passwd, and a remote/internal URL payload targeting the loopback address (http://127.0.0.1/). Both were rejected.

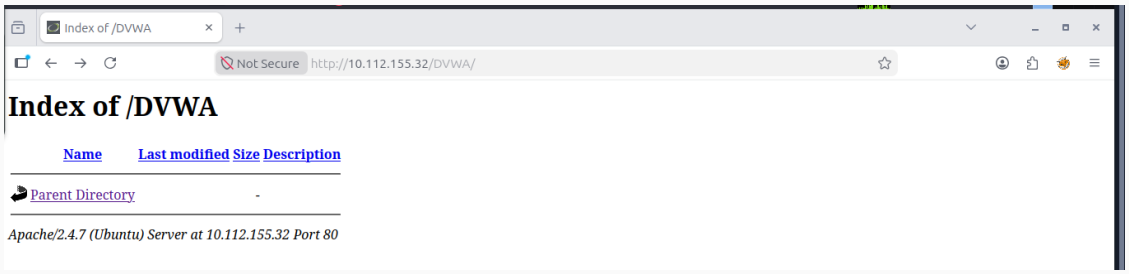
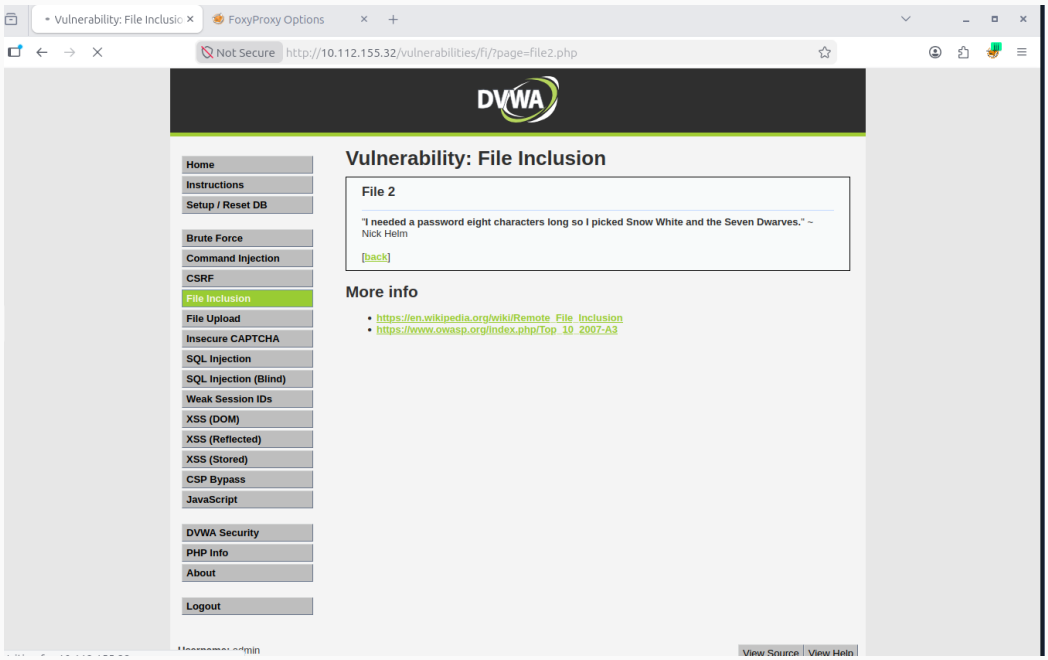


Figure 1 — Apache directory listing confirming the target server stack (reconnaissance).

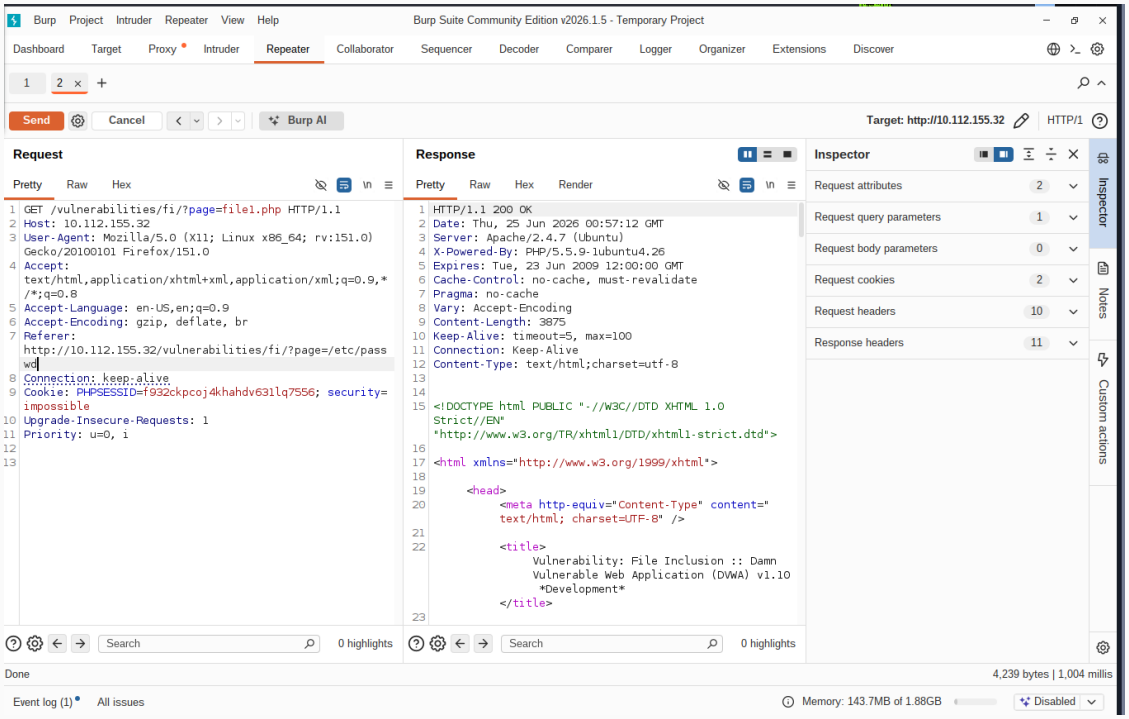
3.6.2 Why the Attempts Failed

At the Impossible level, DVWA validates the page parameter against a strict, fixed allow-list (a PHP in_array() check against the literal values include.php, file1.php, file2.php, file3.php). Any value outside this list — including both payloads used here — is rejected before reaching include(), and the application silently substitutes the default page. Both requests returned HTTP/1.1 200 OK with ordinary File Inclusion content rather than any sign of an outbound or internal connection.

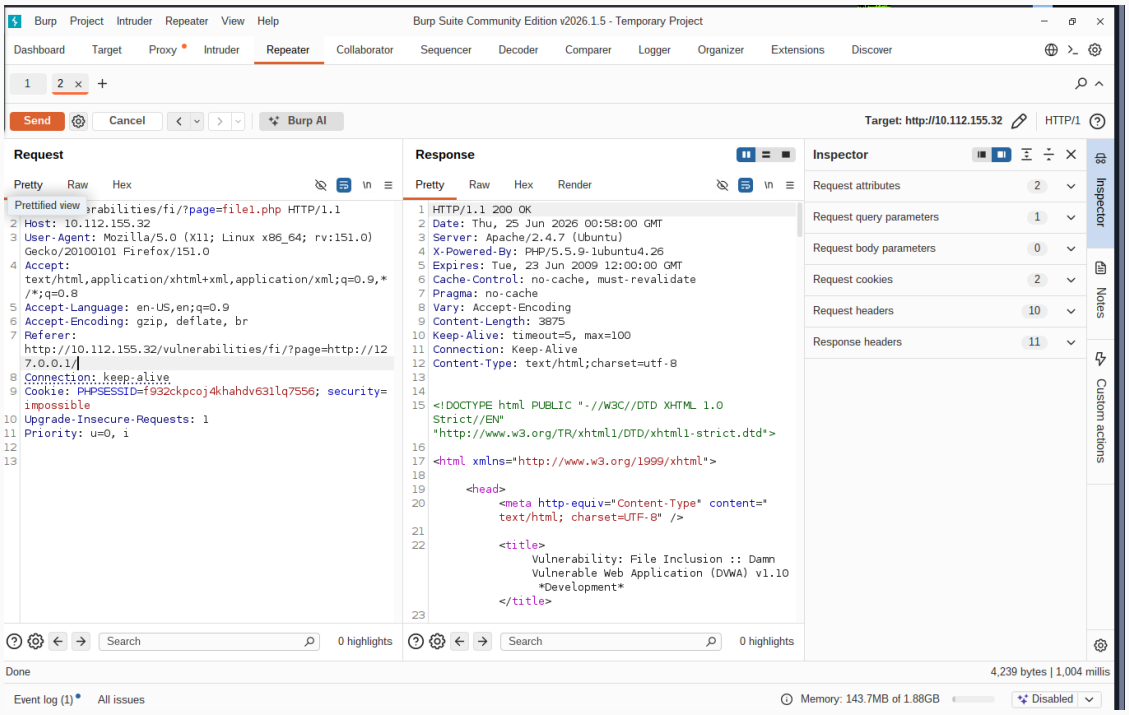
```
Attempt 1 — Local traversal:    ?page=/etc/passwd           → rejected, default page
returned
Attempt 2 — Loopback inclusion: ?page=http://127.0.0.1/     → rejected, default page
returned
```



Browser rendering the default File Inclusion page after the injected value was rejected.



Burp Suite Repeater — local path-traversal attempt against /etc/passwd (Impossible level).



Burp Suite Repeater — internal/loopback URL attempt against http://127.0.0.1/ (Impossible level).

3.6.3 Assessment Limitation

This assessment exercised only the Impossible level. The Low and Medium levels were not tested but are, based on the equivalent findings observed in the other five modules of this report, very likely to be exploitable — DVWA's Low level is expected to apply no validation at all, and the Medium level's blocklist-style filtering follows the same pattern that was bypassed elsewhere in this engagement (Command Injection, CSRF, XSS, Path Traversal). This module should be treated as a partial assessment pending follow-up testing of the remaining levels.

3.6.4 Remediation

Description	Impact	Remediation
Impossible level's strict allow-list is the effective, recommended control and should remain in place.	No SSRF impact achieved in this assessment.	Maintain the allow-list pattern; do not regress to blocklist filtering as used at Low/Medium.
Remote file inclusion remains possible at the PHP configuration level regardless of application logic.	A future application-level gap could still result in SSRF via include()/require().	Set allow_url_include = Off in php.ini to remove the underlying vector entirely.
No network-layer egress controls confirmed.	If an application-level SSRF is found in a different feature, internal services remain reachable.	Apply egress filtering so the application server cannot reach internal-only or loopback addresses.

- Follow-up testing of the Low, Medium, and High security levels is recommended to fully characterise this vulnerability class for the application.

4. Consolidated Risk Register

The table below consolidates the worst-case finding from each module assessed in this engagement, normalised to a common severity scale for prioritisation purposes.

Module	CWE	Severity	Levels Exploitable	Primary Impact
SQL Injection (Blind)	CWE-89	Critical	Low / Medium / High	Full database compromise; all credentials cracked
OS Command Injection	CWE-78	Critical	Low / Medium / High	Arbitrary OS command execution
CSRF	CWE-352	Critical (Low)	Low / Medium / High	Unauthorised account password change
Reflected XSS	CWE-79	High	Low / Medium / High	Arbitrary JS execution; session-cookie theft PoC
Path Traversal	CWE-22	High (CVSS 7.5)	Low / Medium / High	Sensitive file disclosure; credential & RCE escalation path
SSRF	CWE-918	Informational	None confirmed (Impossible only)	No impact demonstrated; Low/Medium/High untested

4.1 Cross-Cutting Observation

Of the eighteen individual level/module combinations exercised across the five fully-tested modules (3 levels × 5 modules), seventeen were successfully exploited. The single resisting control observed in this engagement — the SSRF/File Inclusion Impossible-level allow-list — was also the only control in the entire assessment built on positive validation (allow-listing) rather than negative validation (blacklisting/blocklisting). This strongly supports allow-listing as the architecturally preferred control for all of the input-validation classes covered in this report.

5. Consolidated Recommendations

Recommendations are grouped by theme and ordered by the number of modules they would remediate, reflecting the highest-leverage fixes first.

5.1 Replace Blacklisting with Allow-Listing or Parameterisation (affects 5 of 6 modules)

- SQL Injection: use parameterised queries / prepared statements universally.
- Command Injection: avoid shell invocation; where unavoidable, whitelist numeric IPv4 octets only.
- Reflected XSS: apply context-aware output encoding (htmlspecialchars) rather than tag-name blacklists or regexes.
- Path Traversal: restrict file parameters to an explicit allow-list and enforce canonical path checks with realpath().
- SSRF: retain and extend the allow-list pattern already proven effective at the Impossible level.

5.2 Harden Session & Request-Binding Controls (CSRF, XSS, SQLi)

- Implement a unique, cryptographically random, per-session anti-CSRF token on every state-changing request, with strict one-time-use (nonce) enforcement.
- Do not rely on the Referer header alone for origin verification.
- Set HttpOnly and Secure flags on all session cookies to blunt the impact of any residual XSS.
- Deploy a strict Content-Security-Policy restricting script-src to the application's own origin.

5.3 Reduce Blast Radius (Defence in Depth)

- Apply the principle of least privilege to the database account used by the application (no INSERT/UPDATE/DELETE/DROP where not required).
- Run the web server under a dedicated low-privilege account and apply PHP open_basedir restrictions.
- Disable allow_url_include and allow_url_fopen to remove remote/stream-wrapper inclusion vectors.
- Apply network-layer egress filtering to prevent the application server from reaching internal-only or loopback addresses.
- Deploy a Web Application Firewall (e.g. ModSecurity with the OWASP Core Rule Set) as an additional detection and blocking layer across all modules.

5.4 Platform & Cryptographic Hygiene

- Upgrade PHP from 5.5.9 (end-of-life since July 2016) to a currently supported 8.2+ release, and keep MySQL/MariaDB current.
- Replace unsalted MD5 password hashing with bcrypt, Argon2, or PBKDF2 using a unique per-user salt.

5.5 Follow-Up Testing

- Complete SSRF testing at the Low, Medium, and High security levels to fully characterise that vulnerability class.
- Re-test all six modules after remediation to confirm closure, including regression testing of the High/Impossible levels to ensure fixes do not introduce new bypasses.

6. Conclusion

Taken together, these six assessments demonstrate that DVWA's progressively hardened security levels accurately model the real-world maturity curve of input-validation defences. Low-level implementations, which apply no validation whatsoever, were uniformly and trivially exploitable. Medium-level implementations, which introduce blacklist- or substring-based filtering, were bypassed in every module tested, underscoring that blacklisting is fundamentally reactive and incomplete regardless of how comprehensive it appears. High-level implementations fared better in absolute terms but were still bypassed in four of five modules, typically through a narrow, precise gap in an otherwise more rigorous control — an unfiltered operator variant, a regex that didn't anticipate a non-‘script’ vector, a prefix check that didn't validate the full resolved path, or a token that wasn't invalidated after use.

The sole exception was the SSRF assessment's Impossible-level finding, where a strict, fixed allow-list correctly rejected every payload attempted. This is consistent with, and reinforces, the consolidated recommendation in Section 5.1: wherever an application can enumerate the closed set of values it actually needs to accept, allow-listing should be preferred over any form of blacklist, blocklist, or pattern-based filtering.

Because the SSRF module was assessed only at its strongest setting, this report should be read as a complete picture for five of the six modules and a partial picture for SSRF specifically. Completing SSRF testing at the Low, Medium, and High levels — and re-testing all modules after the remediations in Section 5 are implemented — is recommended as the immediate next step for this engagement.

Appendix A — Tools, References & Source Reports

A.1 Tools Used Across This Engagement

- Burp Suite (Community and Professional editions) — proxy interception, Repeater-based request replay
- SQLMap v1.9.8 — automated blind SQL injection enumeration and data extraction
- Firefox — manual browser-based testing and payload submission

A.2 External References

- OWASP Top 10 (2021) — https://owasp.org/Top10/
- OWASP — Path Traversal: https://owasp.org/www-community/attacks/Path_Traversal
- OWASP — Cross-Site Request Forgery (CSRF): https://owasp.org/index.php/Cross-Site_Request_Forgery
- OWASP — OS Command Injection reference
- MITRE CWE-22, CWE-78, CWE-79, CWE-89, CWE-352, CWE-918

A.3 Source Reports Consolidated in This Document

Module	Original Report Title	Assessor	Date
SSRF	DVWA SSRF Pentest Report	Yousef Gamal Sleem	25 June 2026
SQL Injection (Blind)	DVWA SQLi (Blind) Pentest Report	Abdalrhman	24–25 June 2026
OS Command Injection	DVWA OS Command Injection Report	Group 3 (DEPI)	22–23 June 2026
CSRF	DVWA CSRF Report	Saher Tarek	25–26 June 2026
Reflected XSS	DVWA Reflected XSS Pentest Report	Mohamed Abuzahda	26 June 2026
Path Traversal	DVWA Path Traversal Pentest Report	Moamen	24–25 June 2026